

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
"НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
"ВЫСШАЯ ШКОЛА ЭКОНОМИКИ"
ФАКУЛЬТЕТ КОМПЬЮТЕРНЫХ НАУК**

Никитюк Михаил Андреевич

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

**Разработка и внедрение комплексного сервиса для управления бронированием
ассистентов и автоматической поддержки студентов с использованием RAG
технологий**

по направлению подготовки 09.04.02 Прикладная математика и информатика
образовательная программа «Аналитика больших данных»

Студент

М. А. Никитюк

Научный руководитель

Академический руководитель

ОП:Аналитика больших данных

Е. А. Паточенко

Москва 2026

Содержание

Аннотация	4
1 Введение	5
2 Теоретическое обоснование принципа работы программы и обзор существующих решений	12
2.1 Введение в проблему автоматизации бронирования ассистентов	12
2.2 Обзор существующих решений и их ограничений	14
2.3 Обоснование выбора разрабатываемого подхода	18
2.4 Выводы по первой главе	21
3 Постановка задачи и требования к системе	23
3.1 Цель и задачи разработки	24
3.2 Описание ролей пользователей	24
3.3 Функциональные требования	25
3.4 Нефункциональные требования	28
3.5 Требования к RAG-помощнику	29
3.6 Выводы по второй главе	29
4 Проектирование архитектуры системы	32
4.1 Общая архитектура сервиса	32
4.2 Архитектура клиентской части	32
4.3 Архитектура серверной части основного приложения	32
4.4 Структура базы данных	32
4.5 Архитектура RAG-модуля	32
4.6 Развертывание системы	32
4.7 Выводы по третьей главе	32
5 Реализация разработанного сервиса	32
5.1 Реализация интерфейса менеджера	32
5.2 Реализация интерфейса преподавателя	32
5.3 Реализация интерфейса студента	32
5.4 Реализация механизма заявок и выбора ассистентов	32
5.5 Реализация RAG-помощника	32

5.6	Интеграция компонентов системы	32
5.7	Выводы по четвертой главе	32
6	Тестирование и оценка разработанного решения	32
6.1	Подход к тестированию	32
6.2	Проверка пользовательских сценариев	32
6.3	Оценка работы RAG-помощника	32
6.4	Сравнение с исходным процессом на основе таблиц	32
6.5	Сравнение с существующими аналогами	32
6.6	Выводы по пятой главе	32
7	Заключение	32
	Список литературы	32
	Приложения	32

Аннотация

В данной работе рассматривается разработка и внедрение комплексного сервиса для управления бронированием ассистентов и автоматической поддержки студентов с использованием RAG-технологий. Актуальность работы связана с тем, что подбор ассистентов, согласование заявок, оформление документов и поиск информации по внутренним регламентам часто выполняются вручную или с помощью больших табличных файлов. Такой подход усложняет работу преподавателей, студентов и менеджеров, повышает риск ошибок и создает административную нагрузку.

Целью работы является создание веб-сервиса, который заменяет разрозненные таблицы единой системой с разграничением пользовательских ролей и встроенным интеллектуальным помощником. В рамках проекта выделены три основные роли: преподаватель, студент и менеджер. Для каждой роли определены основные сценарии взаимодействия, после чего реализованы соответствующие интерфейсы и серверная логика.

Сервис построен на клиент-серверной архитектуре. Клиентская часть реализована с использованием `React` и `Tailwind CSS`, серверная часть основного веб-приложения построена на базе `Node.js` и `Express.js`, а для хранения данных используется `PostgreSQL`. Отдельный сервер и вычисления RAG-помощника реализованы на языке `Python`. Для развертывания применяются `Docker` и `Docker Compose`. RAG-помощник работает на основе локальной языковой модели и формирует ответы с опорой на внутренние положения, инструкции по оплатам, регламенты бронирования, шаблоны договоров и FAQ.

В результате был разработан прототип системы, объединяющий бронирование ассистентов, управление заявками, роль менеджера и автоматический поиск ответов по документам. Полученное решение может использоваться для снижения нагрузки на менеджеров, повышения прозрачности процессов и упрощения взаимодействия между участниками образовательного процесса.

Ключевые слова

RAG-технологии, веб-приложение, бронирование ассистентов, `PostgreSQL`, `React`, `Node.js`, `Express.js`, образовательные сервисы, автоматизация административных процессов.

1 Введение

Современные образовательные организации представляют собой сложные системы, включающие множество участников, административных процессов и информационных потоков. В рамках образовательного процесса взаимодействуют преподаватели, студенты, менеджеры образовательных программ, учебные офисы и другие подразделения, обеспечивающие организацию занятий, сопровождение курсов, контроль учебной нагрузки и выполнение внутренних регламентов. Качество такого взаимодействия напрямую влияет на то, насколько удобно участникам образовательного процесса выполнять свои задачи. Если действия преподавателей, студентов и менеджеров плохо согласованы между собой, то даже простой на первый взгляд процесс может приводить к задержкам, ошибкам в документах, дополнительным уточнениям и росту административной нагрузки. Поэтому для образовательной организации важно не только обеспечить саму возможность взаимодействия между участниками, но и сделать этот процесс понятным, прозрачным и управляемым. В случае отсутствия специализированного инструмента данный процесс может становиться трудоемким и плохо масштабируемым.

На практике подобные процессы часто реализуются с использованием табличных инструментов, таких как Excel или Google Sheets. Несмотря на универсальность и доступность данных решений, их применение для управления бронированием ассистентов имеет ряд существенных ограничений. Таблицы плохо подходят для одновременной работы большого количества пользователей с различными ролями, не обеспечивают удобного разграничения прав доступа, затрудняют контроль статусов заявок и не позволяют в явном виде формализовать бизнес-логику процесса. Кроме того, при увеличении числа преподавателей, студентов и бронирований возрастает риск ошибок, дублирования записей, потери актуальности данных и возникновения дополнительных административных издержек.

Проблема усложняется тем, что взаимодействие между участниками процесса не ограничивается только выбором ассистента. В процессе работы у пользователей регулярно возникают вопросы, связанные с правилами бронирования, порядком оформления документов, механизмом формирования выплат, нормативными ограничениями, сроками подачи заявок и иными административными аспектами. На практике ответы на такие вопросы редко находятся в одном месте. Часть информации содержится во внутренних положениях, часть - в инструкциях по оплатам, часть - в регламентах бронирования, шаблонах договоров или отдельных списках часто задаваемых вопросов. Из-за этого пользователю приходится либо самостоятельно просматривать несколько документов, либо обращаться к менеджеру за

уточнением. При этом сложность состоит не только в объеме документов, но и в том, что пользователь не всегда заранее понимает, в каком именно документе находится нужное правило и какими словами оно сформулировано.

Именно поэтому в рамках данной работы возникла идея не ограничиваться только интерфейсом для бронирования ассистентов, а разработать более комплексный сервис. Такой сервис должен объединять два направления: управление заявками на ассистентов и помощь пользователям при поиске ответов на административные вопросы. В этом случае преподаватель, студент или менеджер получает не просто форму для работы с бронированиями, а единую среду, в которой можно выполнять основные действия и сразу уточнять связанные с ними правила. Для реализации такой поддержки могут использоваться технологии retrieval-augmented generation, далее RAG. Их основная идея заключается в том, что языковая модель получает не только сам вопрос пользователя, но и дополнительные фрагменты из заранее подготовленной базы документов, на которые она может опираться при формировании ответа.

Подход RAG хорошо подходит для задач, где важно отвечать не в общем виде, а с учетом конкретных внутренних материалов организации. Обычный чат-бот может дать правдоподобный ответ, но при этом не всегда понятно, на какой источник он опирается и соответствует ли этот ответ действующим правилам. В RAG-системе перед генерацией ответа выполняется поиск по корпусу документов, после чего найденные фрагменты передаются модели как контекст. Благодаря этому ответ становится более связанным с исходными материалами, а риск появления неточных или слишком общих формулировок снижается. В рассматриваемом проекте такой механизм может использоваться для работы с инструкциями по оплатам, правилами бронирования, шаблонами договоров, FAQ и другими документами, с которыми регулярно сталкиваются преподаватели, студенты и менеджеры.

Актуальность данной работы связана с тем, что ручное сопровождение процесса бронирования ассистентов плохо масштабируется. Пока количество заявок невелико, таблиц и переписка может быть достаточно, однако при росте числа преподавателей, студентов и административных условий такой способ работы становится неудобным. Менеджерам приходится чаще проверять данные, отвечать на повторяющиеся вопросы и следить за тем, чтобы участники процесса использовали актуальные правила. При этом современные средства веб-разработки и локальные языковые модели позволяют создавать более специализированные решения, ориентированные на конкретную предметную область. В отличие от универсальных сервисов, такая система может учитывать реальные роли пользователей, внутреннюю логику бронирования, особенности хранения данных и требования образовательной органи-

зации.

В рамках данной работы рассматривается разработка и внедрение комплексного сервиса для управления бронированием ассистентов и автоматической поддержки студентов с использованием RAG-технологий. Проект является развитием ранее созданного прототипа, предназначенного для замены неудобного процесса работы с большими таблицами. В новой версии системы основное внимание уделяется доработке визуальной части приложения, добавлению роли менеджера, расширению пользовательских сценариев и внедрению RAG-помощника, предназначенного для автоматической поддержки пользователей при работе с внутренними документами.

Разрабатываемая система предполагает наличие трех основных ролей: преподавателя, студента и менеджера. Преподаватель получает возможность создавать и просматривать бронирования ассистентов, выбирать подходящих студентов и отслеживать состояние заявок. Студент получает возможность взаимодействовать с системой как потенциальный ассистент, просматривать релевантную информацию и получать ответы на вопросы, связанные с участием в процессе ассистирования. Менеджер, в свою очередь, получает инструменты для контроля процесса, обработки заявок, сопровождения пользователей и администрирования данных. RAG-помощник в такой структуре выступает как общий инструмент поддержки, доступный для всех трех ролей. Его задача состоит в том, чтобы пользователь мог не искать нужный документ вручную, а задать вопрос в свободной форме и получить ответ, основанный на внутренних инструкциях, положениях и регламентах.

Техническая часть проекта строится вокруг классической клиент-серверной архитектуры. Интерфейс приложения реализован с использованием `React` и `Tailwind CSS`. Такой выбор был сделан из-за того, что данные инструменты позволяют достаточно быстро разрабатывать визуальные компоненты, переиспользовать их в разных частях приложения и адаптировать интерфейс под различные пользовательские сценарии.

Основная серверная часть веб-приложения реализована на `Node.js` с использованием фреймворка `Express.js`. Она отвечает за обработку запросов от клиента, выполнение основной бизнес-логики и обмен данными с базой данных. В качестве системы хранения используется `PostgreSQL`, поскольку в проекте необходимо работать со связанными сущностями: пользователями, ролями, заявками, бронированиями и административными статусами. Для запуска отдельных компонентов системы применяется `Docker` и `Docker Compose`, что упрощает настройку окружения и делает развертывание проекта более воспроизводимым.

Отдельно от основной серверной части реализован модуль RAG-помощника. Он написан на языке `Python`, так как большая часть инструментов для работы с языковыми моде-

лями, обработкой текстов и векторным поиском удобнее доступна именно в этой экосистеме. Вычисления RAG-модуля выполняются с использованием локальной языковой модели. Такой подход был выбран для того, чтобы не передавать внутренние документы во внешние сервисы и сохранить больший контроль над обработкой данных внутри проекта.

Важной особенностью работы является то, что разработка системы выполнялась в условиях необходимости самостоятельного освоения полного цикла веб-разработки. В ходе выполнения проекта были изучены принципы построения клиент-серверных приложений, проектирования интерфейсов, организации серверной логики, работы с базами данных, контейнеризации и интеграции языковых моделей. При изучении отдельных технически сложных конструкций использовались инструменты искусственного интеллекта, которые применялись как вспомогательное средство для объяснения концепций, анализа ошибок и ускорения освоения новых технологий. При этом проектирование, реализация и адаптация системы под предметную область выполнялись автором самостоятельно.

Цель исследования:

Целью данной работы является разработка и внедрение комплексного сервиса для управления бронированием ассистентов и автоматической поддержки пользователей с использованием RAG-технологий, направленного на повышение удобства взаимодействия преподавателей, студентов и менеджеров в рамках образовательного процесса.

Задачи исследования:

1. Проанализировать существующий процесс бронирования ассистентов и выявить основные ограничения использования табличных инструментов для решения данной задачи.
2. Определить роли пользователей системы, их основные сценарии взаимодействия и требования к функциональности сервиса.
3. Разработать архитектуру веб-приложения, включающую клиентскую часть, серверную часть, базу данных и модуль автоматической поддержки пользователей.
4. Реализовать пользовательские интерфейсы для преподавателей, студентов и менеджеров с учетом различий в их задачах и правах доступа.
5. Разработать механизм хранения и обработки данных о пользователях, заявках, бронированиях и связанных административных сущностях.
6. Интегрировать в систему RAG-помощника, использующего внутренние нормативно-правовые положения, инструкции по оплатам, регламенты бронирования, шаблоны договоров и FAQ.
7. Обеспечить возможность развертывания сервиса с использованием Docker и Docker Compose.

8. Провести анализ разработанного решения с точки зрения удобства использования, полноты реализованной функциональности и применимости в реальном образовательном процессе.

Предмет исследования:

Предметом исследования являются методы и подходы к разработке веб-сервисов для автоматизации административных процессов в образовательной организации, а также применение RAG-технологий для автоматической поддержки пользователей при работе с внутренними документами.

Объект исследования:

Объектом исследования является процесс управления бронированием ассистентов преподавателей, включающий взаимодействие преподавателей, студентов и менеджеров, а также сопутствующие административные процедуры, связанные с оформлением, оплатой и нормативным сопровождением ассистирования.

Методы исследования:

1. Анализ существующего процесса бронирования ассистентов и выявление его основных проблем.
2. Анализ требований пользователей и формализация основных ролей в системе.
3. Проектирование архитектуры клиент-серверного веб-приложения.
4. Проектирование структуры базы данных для хранения информации о пользователях, заявках и бронированиях.
5. Реализация веб-сервиса с использованием `React`, `Node.js`, `Express.js`, `Tailwind CSS` и `PostgreSQL`.
6. Интеграция RAG-модуля на основе локальной языковой модели, развернутой на отдельном `Python` сервере.
7. Контейнеризация приложения с использованием `Docker` и `Docker Compose`.
8. Проверка работоспособности системы на основных пользовательских сценариях.

Практическая значимость:

Практическая значимость данной работы заключается в создании сервиса, который может быть использован для упрощения и частичной автоматизации процесса бронирования ассистентов в образовательной организации. Разработанная система позволяет заменить работу с большими таблицами более структурированным веб-интерфейсом, снизить количество ручных действий со стороны менеджеров и повысить прозрачность процесса для преподавателей и студентов. Внедрение RAG-помощника дополнительно позволяет сократить время, необходимое пользователям для поиска информации во внутренних документах, и умень-

шить количество повторяющихся административных обращений.

Теоретическая значимость:

Теоретическая значимость данной работы заключается в рассмотрении подхода к интеграции RAG-технологий в специализированную образовательную информационную систему. В работе анализируется возможность объединения классической клиент-серверной архитектуры, ролевой модели доступа и модуля автоматической поддержки пользователей на основе локальной языковой модели. Полученные результаты могут быть использованы при дальнейшем изучении способов применения языковых моделей в административных процессах образовательных организаций.

Научная новизна данного исследования заключается в следующем:

1. В рамках работы был разработан не отдельный инструмент для закрепления заявок, а комплексный сервис, который объединяет процесс бронирования ассистентов и автоматическую информационную поддержку пользователей.

2. В системе была введена отдельная роль менеджера. Это позволяет перенести административный контроль из внешних таблиц и переписок внутрь самого сервиса, а также сделать процесс обработки заявок более структурированным.

3. Для работы с внутренними документами был реализован RAG-помощник, который использует положения, инструкции, регламенты, шаблоны договоров и FAQ как базу знаний для формирования ответов на вопросы пользователей.

4. Была предложена архитектура, в которой основное веб-приложение, база данных и модуль интеллектуальной поддержки разделены на отдельные компоненты и могут запускаться в контейнеризированной среде.

5. В ходе работы была показана возможность разработки full-stack сервиса, ориентированного на конкретный административный процесс образовательной организации, с последующим расширением его функциональности за счет локальной языковой модели.

Таким образом, результат работы заключается не только в реализации веб-интерфейса для бронирования ассистентов, но и в создании более общей системы, которая помогает формализовать административный процесс и уменьшить количество ручных действий со стороны его участников. Разработанный сервис позволяет повысить удобство взаимодействия между преподавателями, студентами и менеджерами, а также демонстрирует применимость RAG-технологий для поддержки пользователей при работе с внутренними документами организации.

Структура исследования определена введением, главами основной части, заключением, списком использованных источников и приложениями. В первой главе рассматривается

предметная область, существующий процесс бронирования ассистентов и ограничения используемых инструментов. Во второй главе формулируются требования к системе и описываются основные пользовательские роли. В третьей главе рассматривается архитектура и реализация разработанного сервиса. В четвертой главе описывается интеграция RAG-помощника и особенности работы с внутренними документами. В пятой главе приводится оценка разработанного решения и анализ его применимости.

2 Теоретическое обоснование принципа работы программы и обзор существующих решений

2.1 Введение в проблему автоматизации бронирования ассистентов

Современная образовательная среда включает большое количество процессов, связанных не только с проведением занятий и контролем знаний, но и с административным сопровождением учебной деятельности. Одним из таких процессов является организация работы ассистентов преподавателей. Ассистенты могут участвовать в проверке домашних заданий, проведении консультаций, сопровождении проектной деятельности, подготовке учебных материалов и решении других задач, возникающих в рамках образовательного процесса. При этом само назначение ассистента на курс или конкретную задачу является не только учебным, но и административным процессом, так как требует согласования между преподавателем, студентом и менеджером образовательной программы.

На первый взгляд задача бронирования ассистентов может показаться достаточно простой, поскольку она сводится к выбору подходящего студента и фиксации соответствующей договоренности. Однако на практике данный процесс содержит множество дополнительных ограничений. Необходимо учитывать доступность студента, его компетенции, интерес к конкретному курсу, требования преподавателя, количество уже взятых задач, сроки согласования, оформление документов, особенности оплаты и внутренние положения образовательной организации. Кроме того, различные участники процесса обладают разными правами и целями: преподаватель заинтересован в поиске подходящего ассистента, студент — в получении понятной информации о возможностях и условиях работы, а менеджер — в контроле корректности всего процесса.

Если в процессе участвует небольшое количество преподавателей и студентов, то такую работу действительно можно какое-то время вести вручную. В этом случае достаточно общей таблицы, нескольких переписок и набора инструкций, к которым участники обращаются при необходимости. Однако по мере роста числа заявок данный способ начинает создавать все больше трудностей. В таблицах появляются дополнительные поля, статусы и комментарии, отдельные записи могут дублироваться, а актуальное состояние заявки становится не всегда очевидным. Менеджеру приходится постоянно проверять, какие данные уже заполнены, какие заявки требуют действия, кому необходимо ответить и какие правила нужно дополнительно пояснить.

Из-за этого появляется потребность в отдельной программной системе, которая будет

не просто хранить список бронирований, а задавать понятный порядок работы с ними. Такая система должна учитывать, что преподаватель, студент и менеджер выполняют разные действия и видят процесс с разных сторон. Для преподавателя важен выбор подходящего ассистента и состояние его заявки, для студента - понимание доступных возможностей и условий участия, для менеджера - контроль корректности данных и сопровождение процесса. Поэтому в сервисе необходимо явно описать роли пользователей, права доступа, статусы бронирований и действия, которые могут выполняться на каждом этапе.

Еще одна сложность связана с тем, что пользователям часто нужна не только сама форма заявки, но и пояснения к административным правилам. Например, преподавателю может быть важно понять, как правильно оформить бронирование, студенту - какие документы нужно подписать, а менеджеру - быстро найти нужный пункт регламента или инструкцию по выплатам. Эти сведения обычно находятся не в одном документе, а распределены между положениями, инструкциями, шаблонами договоров, регламентами и списками часто задаваемых вопросов. Поэтому даже при наличии всех материалов поиск ответа может занимать время, особенно если пользователь не знает, где именно находится нужная информация.

Для решения такой задачи в работе рассматривается применение технологий retrieval-augmented generation, далее RAG. Идея данного подхода состоит в том, что языковая модель при подготовке ответа использует не только собственные внутренние знания, но и фрагменты документов, найденные по запросу пользователя. Такой принцип был описан Льюисом и соавторами, которые рассматривали RAG как объединение параметрической памяти модели и непараметрической памяти в виде внешнего индекса документов [lewis2020rag]. Для рассматриваемого проекта это особенно важно, так как внутренние документы могут изменяться, а ответы помощника должны быть связаны с актуальными инструкциями и регламентами.

В рамках данной работы RAG-помощник используется как дополнительный модуль сервиса, а не как замена менеджера. Его основная задача заключается в том, чтобы снять часть повторяющихся вопросов и ускорить поиск информации. Пользователь может задать вопрос в свободной форме, после чего система подбирает релевантные фрагменты из внутренней базы документов и формирует ответ на их основе. Такой механизм позволяет объединить в одном сервисе работу с бронированиями и справочную поддержку по положениям, инструкциям по оплатам, правилам оформления, шаблонам договоров и FAQ.

Основные термины:

Веб-сервис - программная система, с которой пользователь работает через веб-интерфейс. В данной работе под веб-сервисом понимается приложение для управления бронированием

ассистентов и взаимодействия между преподавателями, студентами и менеджерами.

Пользовательская роль - набор прав и доступных действий, который определяет, как именно пользователь взаимодействует с системой. В разрабатываемом сервисе используются три роли: преподаватель, студент и менеджер. Такое разделение необходимо для того, чтобы каждая группа пользователей работала только с теми функциями, которые относятся к ее задачам.

Бронирование ассистента - заявка на привлечение студента к выполнению ассистентских задач. В системе такое бронирование имеет участников, статус, связанные сроки и административные данные, необходимые для дальнейшего сопровождения процесса.

Менеджер - пользователь, который отвечает за административное сопровождение бронирований. В отличие от преподавателя и студента, менеджер имеет расширенные возможности для проверки заявок, контроля статусов и поддержки пользователей.

RAG-помощник - модуль системы, который ищет релевантные фрагменты во внутренних документах и использует их для формирования ответа на вопрос пользователя. В данной работе такой помощник применяется для поддержки пользователей при работе с регламентами, инструкциями и другими внутренними материалами образовательной организации.

Локальная языковая модель - модель, вычисления которой выполняются на собственной инфраструктуре проекта, а не через внешний облачный API. Такой подход может быть особенно важен в случаях, когда система работает с внутренними документами и административной информацией.

2.2 Обзор существующих решений и их ограничений

Перед разработкой специализированного сервиса необходимо рассмотреть существующие программные решения, которые частично пересекаются с поставленной задачей. К таким решениям можно отнести табличные инструменты, системы онлайн-бронирования, low-code платформы, образовательные LMS-системы, сервисы для проверки работ и системы автоматической поддержки пользователей. Несмотря на то, что каждое из этих решений может закрывать отдельную часть проблемы, их использование в исходном виде не позволяет полностью решить задачу управления бронированием ассистентов и автоматической поддержки пользователей.

Наиболее простым и распространенным подходом является использование электронных таблиц. Таблицы удобны тем, что позволяют быстро создать структуру данных, от-

крыть доступ нескольким пользователям и вручную редактировать записи. Однако их основное преимущество одновременно становится и ограничением. Таблица не задает строгую бизнес-логику процесса, не обеспечивает полноценного жизненного цикла заявки и не отделяет действия преподавателя, студента и менеджера на уровне интерфейса. В результате контроль корректности данных переносится на пользователей, что увеличивает риск ошибок. Кроме того, при большом числе записей таблицы становятся менее удобными для навигации и анализа.

Другой класс решений составляют сервисы онлайн-бронирования и планирования встреч. Например, **Microsoft Bookings**¹ предназначен для планирования и управления встречами, позволяет пользователям выбирать доступное время и интегрируется с календарем Microsoft 365. Аналогичным образом **Calendly**² предоставляет инструменты для автоматизации назначения встреч, публикации доступных временных слотов, настройки типов событий и напоминаний. Такие сервисы хорошо решают задачу выбора времени, но задача бронирования ассистента шире обычного календарного события. В ней необходимо учитывать не только свободный слот, но и образовательный контекст, роль студента, требования преподавателя, административный статус заявки, возможные ограничения по оплате и документы, связанные с ассистированием. Поэтому использование подобных сервисов не позволяет полностью формализовать процесс.

Существуют также платформы для построения внутренних приложений и рабочих процессов, например **Airtable**³. Данный сервис позиционируется как платформа для создания приложений, управления данными и автоматизации рабочих процессов. По сравнению с обычными таблицами такие инструменты предоставляют больше возможностей для настройки представлений, фильтрации данных и автоматизации отдельных действий. Однако они остаются универсальными платформами, которые требуют дополнительной настройки под конкретный процесс. Для рассматриваемой задачи это означает, что значительная часть предметной логики, включая роли пользователей, обработку заявок, интерфейс менеджера и интеграцию RAG-помощника, должна быть спроектирована поверх базовых возможностей платформы. Кроме того, подобные решения не всегда подходят в случаях, когда требуется полный контроль над хранением данных и вычислениями.

Отдельно необходимо рассмотреть образовательные Отдельно необходимо рассмот-

¹Официальный сайт Microsoft Bookings: <https://www.microsoft.com/en-us/microsoft-365/business/scheduling-and-booking-app>, дата обращения: 25.04.2026.

²Официальный сайт Calendly: <https://calendly.com>, дата обращения: 25.04.2026.

³Официальный сайт Airtable: <https://www.airtable.com>, дата обращения: 25.04.2026.

реть образовательные LMS-системы⁴, такие как Moodle⁵ и Canvas⁶. Moodle является открытой системой управления обучением, предназначенной для создания онлайн-курсов и организации образовательного процесса. Canvas также относится к LMS-системам и применяется для организации курсов, размещения заданий, выставления оценок, сбора обратной связи и коммуникации со студентами. По своей сути такие платформы хорошо подходят для сопровождения уже существующего учебного процесса. Однако задача бронирования ассистентов находится немного в другой плоскости. Здесь важно не только добавить пользователя на курс или выдать ему определенные права, но и провести отдельный административный процесс: выбрать подходящего студента, согласовать его участие, зафиксировать заявку, отследить ее статус и при необходимости передать информацию менеджеру. В стандартной логике LMS такие действия обычно не выделены как самостоятельный процесс, поэтому использовать подобные системы для рассматриваемой задачи без существенной доработки затруднительно.

Еще одним решением, близким к образовательной среде, является Gradescope⁷. Этот сервис в первую очередь ориентирован на проверку работ и организацию оценивания. Он может быть полезен преподавателям и ассистентам на этапе, когда ассистент уже назначен на курс и участвует в проверке заданий или анализе результатов студентов. Однако сам процесс поиска и закрепления ассистента за преподавателем в нем не решается. В Gradescope нет логики согласования бронирования между преподавателем, студентом и менеджером, а также нет встроенной поддержки административных вопросов, связанных с оформлением документов, выплатами и внутренними регламентами. Поэтому данный сервис можно рассматривать скорее как инструмент для последующей работы ассистента, но не как замену разрабатываемой системе. Следовательно, подобный сервис может рассматриваться только как инструмент для отдельного этапа учебного процесса, но не как аналог разрабатываемой системы в целом.

Еще один класс решений связан с автоматической поддержкой пользователей. Например, Zendesk AI agents⁸ предназначены для автоматизации ответов на обращения пользователей в различных каналах поддержки. Notion AI⁹ и корпоративный поиск Notion позво-

⁴LMS, или Learning Management System, - это система управления обучением, предназначенная для размещения учебных материалов, организации курсов, выдачи и проверки заданий, коммуникации между участниками образовательного процесса и отслеживания результатов обучения.

⁵Официальный сайт Moodle: <https://moodle.org>, дата обращения: 25.04.2026.

⁶Официальный сайт Canvas LMS: <https://www.instructure.com/canvas>, дата обращения: 25.04.2026.

⁷Официальный сайт Gradescope: <https://www.gradescope.com>, дата обращения: 25.04.2026.

⁸Документация Zendesk AI agents: <https://support.zendesk.com/hc/en-us/articles/6970583409690-About-AI-agents>, дата обращения: 25.04.2026.

⁹Официальная страница Notion Enterprise Search: <https://www.notion.com/product/enterprise-search>, дата обращения: 25.04.2026.

ляют находить ответы в рабочем пространстве и подключенных приложениях. Эти решения показывают, что автоматизация поиска информации и ответов на вопросы является востребованным направлением. Однако подобные системы обычно являются внешними платформами общего назначения. Они не встроены в предметную модель бронирования ассистентов и не управляют самими заявками, ролями и статусами. Кроме того, использование внешних сервисов может быть нежелательным при работе с внутренними нормативными документами, если требуется локальное хранение данных и выполнение вычислений на стороне проекта.

Таким образом, существующие решения можно разделить на несколько групп. Табличные инструменты обеспечивают простое хранение данных, но плохо подходят для формализации сложного ролевого процесса. Сервисы бронирования и календарного планирования автоматизируют выбор времени, но не учитывают специфику образовательного ассистирования. LMS-системы хорошо поддерживают учебный процесс, но не решают административную задачу бронирования ассистентов. Системы автоматической поддержки пользователей позволяют отвечать на вопросы, но не являются частью специализированного процесса управления заявками. Поэтому ни одно из рассмотренных решений не покрывает одновременно все ключевые требования данной работы.

Разрабатываемый сервис отличается от перечисленных аналогов тем, что объединяет несколько функций в рамках единой предметной системы. С одной стороны, он предоставляет веб-интерфейс для преподавателей, студентов и менеджеров, позволяющий управлять бронированием ассистентов. С другой стороны, он включает RAG-помощника, который использует внутренние документы для ответа на административные вопросы. Кроме того, использование собственной серверной части, базы данных PostgreSQL¹⁰, так как данная система управления базами данных хорошо подходит для работы со связанными сущностями, такими как пользователи, заявки, роли и бронирования и локальной языковой модели позволяет контролировать структуру данных, логику обработки заявок и вычисления, связанные с автоматической поддержкой пользователей.

В следующих разделах данной главы будут рассмотрены основные принципы построения разрабатываемого сервиса, особенности ролевой модели, архитектурные решения и подход к интеграции RAG-помощника. Также будет показано, каким образом выбранная архитектура позволяет устранить ограничения существующих решений и адаптировать систему под конкретные процессы образовательной организации.

¹⁰PostgreSQL - это объектно-реляционная система управления базами данных с открытым исходным кодом. Официальная документация PostgreSQL: <https://www.postgresql.org/docs/current/intro-what-is.html>, дата обращения: 25.04.2026.

2.3 Обоснование выбора разрабатываемого подхода

После рассмотрения существующих решений можно сделать вывод, что ни один из готовых инструментов не закрывает задачу полностью. Табличные инструменты позволяют быстро хранить данные, но не задают строгую логику процесса. LMS-системы хорошо подходят для сопровождения учебных курсов, но не предназначены для отдельного административного сценария бронирования ассистентов. Сервисы календарного планирования помогают согласовывать время, однако не учитывают специфику образовательной организации, статусы заявок, роли пользователей и связанные с этим документы. Системы поддержки пользователей, в свою очередь, могут отвечать на вопросы, но не управляют самим процессом бронирования.

Поэтому в рамках данной работы был выбран подход, основанный на разработке собственного специализированного веб-сервиса. Такой вариант требует больше времени на проектирование и реализацию, однако дает возможность описать предметную область именно в том виде, в котором она задумана на практике. В системе можно явно выделить преподавателей, студентов и менеджеров, задать для них разные права доступа, определить этапы работы с заявками и связать процесс бронирования с внутренними инструкциями и регламентами. В отличие от универсальных платформ, собственный сервис не пытается быть применимым ко всем возможным задачам, а решает более узкую, но конкретную проблему.

При выборе технических инструментов учитывалось несколько факторов. Во-первых, система должна была быть достаточно простой для дальнейшей поддержки и развития. Во-вторых, важно было использовать технологии, для которых существует большое количество документации, примеров и готовых практик. В-третьих, проект разрабатывался одним человеком, поэтому стек должен был позволять быстро переходить от проектирования интерфейса к реализации серверной логики и работе с базой данных. Именно поэтому в качестве основы была выбрана классическая клиент-серверная архитектура.

Клиентская часть реализуется с использованием **React**¹¹. Данный инструмент был выбран потому, что интерфейс системы состоит из большого количества повторяющихся элементов: карточек заявок, списков студентов, форм, кнопок действий, таблиц и диалоговых окон. Использование шаблонизированного подхода позволяет не писать каждый элемент заново, а выделять общие части интерфейса и переиспользовать их в разных разделах приложения. Это особенно важно при наличии нескольких ролей, так как преподаватель, студент и менеджер могут видеть разные страницы, но многие элементы интерфейса остаются

¹¹**React** - JavaScript-библиотека для разработки пользовательских интерфейсов на основе компонентного подхода. Официальная документация React: <https://react.dev/>, дата обращения: 25.04.2026.

похожими по структуре.

Для стилизации интерфейса используется Tailwind CSS¹². Выбор данного инструмента связан с тем, что он позволяет быстро настраивать внешний вид элементов прямо в разметке, не создавая большое количество отдельных CSS-файлов. Для проекта, в котором важна доработка визуальной части и удобство пользовательского интерфейса, это оказалось практичным решением. Кроме того, такой подход упрощает поддержку единого визуального стиля: одинаковые отступы, размеры, цвета и состояния элементов можно задавать более последовательно.

Основная серверная часть веб-приложения построена на Node.js¹³ и Express.js¹⁴. Этот выбор связан с тем, что данные технологии хорошо подходят для реализации REST API, обработки запросов от клиентской части и организации бизнес-логики. Сервер отвечает за регистрацию и авторизацию пользователей, работу с ролями, получение и изменение данных о заявках, бронированиях и связанных сущностях.

Отдельного внимания требует модуль RAG-помощника. Для его реализации был выбран язык Python¹⁵. Это связано с тем, что именно в Python собрано большое количество библиотек для обработки текстов, работы с языковыми моделями, эмбедингами и поиском по документам. В отличие от основной серверной части, RAG-модуль выполняет не только обработку обычных API-запросов, но и более специфические операции: подготовку документов, разбиение текста на фрагменты, построение поискового индекса, извлечение релевантного контекста и формирование ответа с использованием локальной языковой модели.

Разделение основного сервера и RAG-модуля на разные части является осознанным архитектурным решением. Основное приложение на Node.js отвечает за пользовательские сценарии, работу с интерфейсом и базой данных, а сервер на Python отвечает за интеллектуальную обработку запросов, что позволяет сделать систему более гибкой. Например, при необходимости можно дорабатывать RAG-помощника, менять модель или способ поиска по документам, не переписывая при этом основную логику сервиса бронирования.

Использование локальной языковой модели также связано со спецификой предметной

¹²Tailwind CSS - CSS-фреймворк, основанный на использовании утилитарных классов для быстрой настройки внешнего вида элементов интерфейса. Официальная документация Tailwind CSS: <https://tailwindcss.com/docs>, дата обращения: 25.04.2026.

¹³Node.js - среда выполнения JavaScript вне браузера, которая может использоваться для разработки серверных приложений. Официальная документация Node.js: <https://nodejs.org/api/documentation.html>, дата обращения: 25.04.2026.

¹⁴Express.js - минималистичный веб-фреймворк для Node.js, предназначенный для создания веб-приложений и API. Официальная документация Express.js: <https://expressjs.com/>, дата обращения: 25.04.2026.

¹⁵Python - высокоуровневый язык программирования общего назначения, широко используемый для разработки серверных приложений, обработки данных и работы с моделями машинного обучения. Официальная документация Python: <https://docs.python.org/3/>, дата обращения: 25.04.2026.

области. Внутренние положения, инструкции, шаблоны договоров и документы, связанные с оплатами, могут содержать персональную информацию, которую нежелательно передавать во внешние сервисы. Локальный запуск модели позволяет лучше контролировать обработку данных и снижает зависимость проекта от сторонних API. При этом такой подход имеет и свои ограничения: локальная модель может требовать дополнительных вычислительных ресурсов, а качество ее ответов необходимо проверять отдельно. Однако для рассматриваемой задачи преимущество в виде контроля над данными является достаточно существенным.

Для развертывания системы используются **Docker**¹⁶ и **Docker Compose**¹⁷. Это решение было выбрано для того, чтобы упростить запуск проекта и сделать окружение более воспроизводимым.

Итоговая версия сервиса будет размещена на виртуальной машине, на которой будут настроены необходимые открытые порты для работы клиентской и серверной частей приложения. Доступ пользователей к системе будет осуществляться по протоколу HTTPS, что позволит обеспечить защищенную передачу данных между браузером пользователя и сервером.

В качестве базы данных используется **PostgreSQL**. Данный выбор обусловлен тем, что в системе присутствует большое количество связанных сущностей: пользователи, роли, заявки, бронирования, статусы, документы и история изменений. Реляционная база данных хорошо подходит для такого типа информации, поскольку позволяет явно описывать связи между таблицами и контролировать целостность данных. Для рассматриваемой задачи это важно, так как ошибка в данных может привести не только к некорректному отображению интерфейса, но и к неправильной обработке административного процесса.

Таким образом, выбранный технический подход связан не только с удобством разработки, но и с особенностями самой задачи. Разрабатываемый сервис должен одновременно поддерживать понятный пользовательский интерфейс, ролевую модель, надежное хранение связанных данных, отдельную административную логику и модуль интеллектуальной поддержки. Комбинация **React**, **Tailwind CSS**, **Node.js**, **Express.js**, **PostgreSQL**, **Python**, локальной языковой модели и контейнеризации позволяет разделить эти задачи между разными компонентами и при этом сохранить систему достаточно простой для дальнейшего развития.

¹⁶**Docker** - платформа для разработки, доставки и запуска приложений в контейнерах. Официальная документация Docker: <https://docs.docker.com/>, дата обращения: 25.04.2026.

¹⁷**Docker Compose** - инструмент для описания и запуска многоконтейнерных приложений с помощью единого конфигурационного файла. Официальная документация Docker Compose: <https://docs.docker.com/compose/>, дата обращения: 25.04.2026.

2.4 Выводы по первой главе

В первой главе была рассмотрена предметная область, связанная с организацией бронирования ассистентов преподавателей. Было показано, что данный процесс нельзя рассматривать только как простую запись студента в таблицу или назначение его на курс. На практике он включает несколько участников, различные права доступа, согласование заявок, контроль статусов, работу с документами и необходимость учитывать внутренние правила образовательной организации.

Также были рассмотрены существующие решения, которые частично пересекаются с задачей данной работы. Электронные таблицы удобны на начальном этапе, однако при увеличении числа пользователей и заявок становятся менее прозрачными и требуют большого количества ручных действий. Сервисы календарного бронирования позволяют согласовывать время, но не учитывают образовательный контекст и административную логику процесса. LMS-системы помогают сопровождать учебные курсы, но не решают задачу выбора, согласования и оформления ассистентов. Отдельные системы поддержки пользователей могут отвечать на вопросы, но не связаны напрямую с процессом бронирования.

На основании этого был сделан вывод о необходимости разработки собственного специализированного сервиса. Такой сервис должен учитывать роли преподавателя, студента и менеджера, поддерживать работу с заявками и обеспечивать более понятный порядок взаимодействия между участниками. При этом важной частью системы является не только интерфейс для бронирования, но и механизм информационной поддержки, так как значительная часть вопросов пользователей связана с внутренними положениями, инструкциями, шаблонами договоров и правилами оплаты.

В качестве одного из способов решения этой задачи был рассмотрен RAG-подход. Его применение позволяет связать ответы помощника с конкретными внутренними документами, а не ограничиваться общими формулировками. Для данной работы это особенно важно, поскольку пользователям требуется получать информацию, соответствующую актуальным регламентам и инструкциям образовательной организации.

Кроме того, в главе был обоснован выбор технических инструментов, используемых при разработке. Клиентская часть на `React` и `Tailwind CSS` позволяет реализовать удобный интерфейс для разных ролей пользователей. Серверная часть на `Node.js` и `Express.js` отвечает за обработку основных пользовательских сценариев и бизнес-логику. Для хранения связанных данных используется `PostgreSQL`. RAG-модуль вынесен в отдельный сервер на `Python`, так как именно эта экосистема лучше подходит для работы с текстами, языковы-

ми моделями и поиском по документам. Использование **Docker** и **Docker Compose** позволяет упростить развертывание системы и сделать запуск проекта более воспроизводимым.

Таким образом, по результатам первой главы можно сделать вывод, что разрабатываемый сервис занимает промежуточное положение между системой бронирования, административной панелью и инструментом автоматической поддержки пользователей. Именно объединение этих функций в одном решении отличает проект от рассмотренных аналогов и определяет дальнейшую постановку задачи.

3 Постановка задачи и требования к системе

После анализа предметной области и существующих решений необходимо перейти к более точному и формальному описанию того, какую именно задачу должна решать разрабатываемая система. В первой главе было показано, что процесс бронирования ассистентов нельзя полностью перенести в готовые универсальные сервисы без потери части логики. Поэтому в рамках данной работы разрабатывается отдельный программный продукт, ориентированный на конкретный административный процесс образовательной организации.

Основная идея системы заключается в том, чтобы заменить разрозненную работу с таблицами, переписками и отдельными инструкциями единым веб-сервисом. Пользователь должен иметь возможность войти в систему, увидеть доступные для своей роли действия, выполнить необходимые операции с бронированиями и при необходимости получить справочную информацию через встроенного RAG-помощника. При этом система должна быть понятной не только с технической точки зрения, но и с точки зрения реального пользователя, который не обязан понимать внутреннюю структуру базы данных или логику работы серверной части.

Разрабатываемая система должна поддерживать три основные роли: преподавателя, студента и менеджера. Такое разделение выбрано не случайно, так как именно эти участники чаще всего взаимодействуют в процессе бронирования ассистентов. Введение отдельной роли менеджера является важным отличием новой версии сервиса, поскольку в предыдущем варианте значительная часть административных действий оставалась вне системы. Менеджер создает заявки и контролирует корректность процесса, при необходимости помогая решать административные вопросы, преподаватель просматривает назначенные на него заявки и выбирает себе ассистента, студент выступая в качестве потенциального ассистента, может самостоятельно подавать заявку на желаемые группы.

Как уже упоминалось ранее, отдельной частью постановки задачи является внедрение RAG-помощника. Его внедрение связано с тем, что пользователи часто задают вопросы, ответы на которые находятся во внутренних документах. При этом поиск нужной информации вручную может быть неудобным, особенно если документы имеют разную структуру и относятся к разным аспектам процесса. Поэтому система должна не только хранить и обрабатывать данные о бронированиях, но и помогать пользователям ориентироваться в положениях, инструкциях, регламентах, шаблонах договоров и FAQ.

3.1 Цель и задачи разработки

Целью разработки является создание комплексного веб-сервиса для управления бронированием ассистентов и автоматической поддержки пользователей с использованием RAG-технологий.

Для достижения данной цели необходимо решить следующие задачи:

1. Описать основные роли пользователей системы и определить различия между их правами и сценариями работы.
2. Спроектировать структуру данных, необходимую для хранения информации о пользователях, ролях, заявках, бронированиях и связанных административных сущностях.
3. Реализовать клиентскую часть приложения, которая позволит преподавателям, студентам и менеджерам работать с системой через удобный веб-интерфейс.
4. Реализовать серверную часть основного веб-приложения, отвечающую за обработку пользовательских запросов, взаимодействие с базой данных и выполнение бизнес-логики.
5. Добавить отдельную роль менеджера, позволяющую контролировать процесс бронирования, отслеживать состояние заявок и сопровождать пользователей.
6. Реализовать RAG-помощника, который сможет отвечать на вопросы пользователей на основе внутренних документов образовательной организации.
7. Обеспечить хранение данных в PostgreSQL и организовать взаимодействие между основным сервером, базой данных и RAG-модулем.
8. Подготовить проект к развертыванию с использованием Docker и Docker Compose, а также предусмотреть возможность размещения сервиса на виртуальной машине с доступом по протоколу HTTPS.

3.2 Описание ролей пользователей

В разрабатываемой системе выделяются три основные роли: менеджер, преподаватель и студент, который в дальнейшем может стать ассистентом. Такое разделение необходимо для того, чтобы процесс бронирования был понятен для всех участников и при этом не смешивал между собой административные, преподавательские и студенческие действия. Каждый пользователь работает только с той частью системы, которая относится к его задачам.

Менеджер является пользователем, с которого начинается основной процесс работы с курсом. Он создает заявку на ведение курса и указывает в ней ключевую информацию: дисциплину, ответственного преподавателя, факультет, образовательную программу, количество групп, учебные модули и полезные ссылки. После этого менеджер может отслеживать,

как распределяются ассистенты по группам, какие группы еще остаются без ассистентов и какая нагрузка уже закреплена за каждым студентом. Таким образом, роль менеджера связана с общей организацией и контролем процесса.

Преподаватель работает с теми курсами и группами, за которые он назначен ответственным. Для него система должна показывать текущее состояние групп: отдельно те, где ассистент еще не выбран, и отдельно те, где ассистент уже назначен. Основная задача преподавателя в системе заключается в выборе подходящего ассистента из числа зарегистрированных студентов. Такой подход позволяет преподавателю видеть актуальную ситуацию по своим группам без необходимости обращаться к отдельным таблицам или уточнять данные у менеджера.

Студент рассматривается в системе как потенциальный ассистент. Для него важно видеть, в какие группы он уже был выбран, а также какие группы еще доступны для ассистирования. Поэтому в интерфейсе студента предусмотрены два основных направления работы: просмотр собственных групп и поиск свободных групп. В первом случае студент получает информацию о уже назначенных ему курсах, во втором — может самостоятельно найти подходящую группу и отправить заявку на ассистирование.

Для всех трех ролей также доступен раздел с RAG-помощником. Он используется как общий справочный инструмент, к которому можно обратиться при возникновении вопросов о правилах, документах, оплате, сроках или других внутренних процедурах. При этом смысл обращения к помощнику может отличаться в зависимости от роли: менеджеру он помогает быстрее находить нужные пункты регламентов, преподавателю — уточнять порядок бронирования, а студенту — разбираться в условиях участия в ассистировании.

3.3 Функциональные требования

Функциональные требования описывают основные действия, которые должны быть доступны пользователям в системе. В рамках данной работы они связаны с тремя основными направлениями: созданием и сопровождением заявок на ведение курсов, выбором ассистентов для учебных групп и работой студентов с доступными возможностями ассистирования. Отдельным функциональным блоком является RAG-помощник, который должен быть доступен всем ролям.

К основным функциональным требованиям для роли менеджера относятся:

1. Система должна позволять менеджеру создавать заявку на ведение курса.
2. При создании заявки менеджер должен иметь возможность указать дисциплину, на-

пример анализ данных, программирование на Python, цифровую грамотность или машинное обучение.

3. Система должна позволять указывать ответственного преподавателя, факультет и образовательную программу.

4. Система должна поддерживать указание количества учебных групп в рамках заявки.

5. Система должна позволять выбирать учебные модули, в которых будет проходить курс, от первого до четвертого.

6. Система должна позволять добавлять к заявке полезные ссылки, например ссылку на курс в LMS, беседу в мессенджере, папку с материалами или другие ресурсы.

7. После создания заявки менеджер должен иметь возможность видеть группы, в которых ассистент еще не назначен.

8. Менеджер должен иметь возможность видеть группы, в которых ассистент уже выбран.

9. Система должна показывать менеджеру, сколько групп закреплено за каждым ассистентом.

10. Менеджер должен иметь возможность использовать RAG-помощника для поиска информации во внутренних документах.

К основным функциональным требованиям для роли преподавателя относятся:

1. Система должна показывать преподавателю группы, связанные с его курсами.

2. На странице преподавателя группы должны быть разделены на два блока: группы без ассистента и группы, где ассистент уже выбран.

3. Для групп без ассистента преподаватель должен иметь возможность просматривать студентов, зарегистрированных в сервисе.

4. Преподаватель должен иметь возможность выбрать ассистента для конкретной учебной группы из числа зарегистрированных студентов.

5. После выбора ассистента система должна обновлять состояние группы и переносить ее в блок групп с выбранным ассистентом.

6. Преподаватель должен иметь возможность видеть информацию о группах, где ассистент уже назначен.

7. Преподаватель должен иметь возможность использовать RAG-помощника для уточнения правил бронирования, оформления документов и других связанных вопросов.

К основным функциональным требованиям для роли студента относятся:

1. Система должна предоставлять студенту раздел “мои группы”.

2. В разделе “мои группы” студент должен видеть группы, в которые он уже выбран ассистентом.

3. Система должна показывать студенту информацию о дисциплине, преподавателе, факультете, образовательной программе, учебных модулях и полезных ссылках по его группам.

4. Система должна предоставлять студенту раздел поиска.

5. В разделе поиска студент должен иметь возможность видеть группы, в которых ассистент еще не назначен.

6. Студент должен иметь возможность отправить заявку на ассистирование по выбранной группе.

7. Система должна сохранять информацию об отправленной заявке студента.

8. Студент должен иметь возможность использовать RAG-помощника для получения информации о документах, оплате, правилах участия и внутренних регламентах.

К общим функциональным требованиям относятся:

1. Система должна поддерживать регистрацию или добавление пользователей с указанием их роли.

2. Система должна определять доступные действия пользователя на основе его роли.

3. Система должна хранить данные о пользователях, курсах, учебных группах, заявках, назначенных ассистентах и полезных ссылках.

4. Система должна обеспечивать связь между заявкой менеджера, ответственным преподавателем, учебными группами и wybranymi ассистентами.

5. Система должна обновлять данные после действий пользователей, чтобы изменения были видны другим участникам процесса.

6. Система должна предоставлять всем ролям доступ к отдельному разделу с RAG-помощником.

7. RAG-помощник должен принимать вопрос пользователя в свободной форме.

8. RAG-помощник должен выполнять поиск по внутренним документам и формировать ответ на основе найденных фрагментов.

Таким образом, функциональные требования отражают реальный порядок работы с бронированием ассистентов. Менеджер создает основу процесса в виде заявки на курс, преподаватель выбирает ассистентов для групп, а студент может как отслеживать уже назначенные группы, так и самостоятельно искать возможности для ассистирования. RAG-помощник дополняет эти сценарии и помогает всем участникам быстрее получать ответы на вопросы, связанные с внутренними правилами и документами.

3.4 Нефункциональные требования

Нефункциональные требования описывают не отдельные действия пользователя, а общие свойства системы. Для разрабатываемого сервиса они имеют большое значение, поскольку система должна быть не только работоспособной, но и удобной для дальнейшего развития.

К основным нефункциональным требованиям относятся:

1. **Удобство использования.** Интерфейс должен быть понятным для пользователей с разным уровнем технической подготовки. Преподаватель, студент и менеджер должны быстро понимать, какие действия доступны им в системе.

2. **Разделение прав доступа.** Пользователь не должен получать доступ к функциям, которые не относятся к его роли. Это особенно важно для административных действий менеджера.

3. **Надежность хранения данных.** Информация о пользователях, заявках и бронированиях должна храниться в структурированном виде и не зависеть от ручного редактирования таблиц.

4. **Расширяемость.** Архитектура системы должна позволять добавлять новые функции без полного переписывания проекта. Например, в дальнейшем может потребоваться добавить новые статусы заявок, дополнительные документы или новые сценарии работы менеджера.

5. **Воспроизводимость развертывания.** Проект должен запускаться в заранее описанном окружении, чтобы снизить вероятность ошибок при переносе на другую машину или сервер.

6. **Контроль обработки данных.** Поскольку система работает с внутренними документами и административной информацией, важно по возможности ограничить передачу данных во внешние сервисы. Именно поэтому RAG-модуль ориентирован на использование локальной языковой модели.

7. **Защищенность доступа.** При размещении сервиса на виртуальной машине доступ пользователей должен осуществляться по протоколу **HTTPS**, а необходимые сетевые порты должны быть настроены явно.

Таким образом, нефункциональные требования в данной работе связаны не только с технической надежностью, но и с удобством реального использования системы в образовательной организации.

3.5 Требования к RAG-помощнику

RAG-помощник является отдельным компонентом системы, поэтому к нему предъявляется набор специальных требований. В отличие от обычного справочного раздела, помощник должен работать с вопросами, сформулированными пользователем в свободной форме. Это важно, так как преподаватель, студент или менеджер не всегда знает точное название документа или раздела, в котором находится нужная информация.

К основным требованиям к RAG-помощнику относятся:

1. Помощник должен принимать текстовый вопрос пользователя.
2. Помощник должен выполнять поиск релевантных фрагментов во внутренней базе документов.
3. Ответ должен формироваться с учетом найденного контекста, а не только на основе общих знаний языковой модели.
4. В качестве источников должны использоваться внутренние нормативно-правовые положения, инструкции по оплатам, регламенты бронирования, шаблоны договоров и FAQ.
5. RAG-модуль должен быть реализован отдельно от основного сервера веб-приложения.
6. Вычисления RAG-модуля должны выполняться на стороне проекта с использованием локальной языковой модели.
7. Система должна допускать обновление базы документов без необходимости полной переработки основного веб-приложения.
8. Ответы помощника должны быть ориентированы на практическую помощь пользователю, а не на абстрактное рассуждение.

Для данной работы важно, что RAG-помощник не рассматривается как полностью автономный заменитель менеджера. Он выполняет вспомогательную функцию и помогает пользователю быстрее найти нужную информацию. Если вопрос требует административного решения или проверки конкретной ситуации, окончательное действие все равно остается за ответственным сотрудником. Такой подход позволяет использовать преимущества языковых моделей, но при этом не переносить на них полный контроль над процессом.

3.6 Выводы по второй главе

Во второй главе была сформулирована постановка задачи и описаны основные требования к разрабатываемой системе. Было показано, что сервис должен объединять несколько направлений: управление бронированием ассистентов, поддержку разных пользовательских ролей, хранение связанных данных и автоматическую справочную поддержку с использова-

нием RAG-технологий.

Были выделены три основные роли пользователей: преподаватель, студент и менеджер. Для каждой роли были описаны основные действия и особенности взаимодействия с системой. Такое разделение необходимо для того, чтобы сервис отражал реальный процесс бронирования ассистентов и не смешивал пользовательские и административные функции.

Также были определены функциональные и нефункциональные требования. Функциональные требования задают основные возможности системы, включая работу с заявками, ролями, статусами и RAG-помощником. Нефункциональные требования описывают удобство использования, разделение прав доступа, надежность хранения данных, расширяемость, воспроизводимость развертывания и контроль обработки данных.

Отдельно были сформулированы требования к RAG-помощнику. Было отмечено, что данный модуль должен использовать внутренние документы образовательной организации и помогать пользователям находить ответы на административные вопросы. При этом помощник рассматривается как инструмент поддержки, а не как полная замена менеджера.

Таким образом, во второй главе была подготовлена основа для дальнейшего проектирования архитектуры системы. На следующем этапе необходимо описать, каким образом выделенные требования отражаются в структуре приложения, базе данных, серверной логике и пользовательском интерфейсе.

4 Проектирование архитектуры системы

4.1 Общая архитектура сервиса

4.2 Архитектура клиентской части

4.3 Архитектура серверной части основного приложения

4.4 Структура базы данных

4.5 Архитектура RAG-модуля

4.6 Развертывание системы

4.7 Выводы по третьей главе

5 Реализация разработанного сервиса

5.1 Реализация интерфейса менеджера

5.2 Реализация интерфейса преподавателя

5.3 Реализация интерфейса студента

5.4 Реализация механизма заявок и выбора ассистентов

5.5 Реализация RAG-помощника

5.6 Интеграция компонентов системы

5.7 Выводы по четвертой главе

6 Тестирование и оценка разработанного решения

6.1 Подход к тестированию

6.2 Проверка пользовательских сценариев

6.3 Оценка работы RAG-помощника

6.4 Сравнение с исходным процессом на основе таблиц

6.5 Сравнение с существующими³² аналогами